2 → abc
3 → def.

[2, 3]

ad, ae, af, bd, be, bf, cd, ce, cf.

[2, 3, 4, 5]. ⟶ [4, 5].

[2, 3] ⟶ [3].
(d, e, f).

ad
ae
af
bd
be
bf
cd
ce
cf.

[2, 3] ⟶ [2].
(a, b, c)

ad
bd
cd
ae
be
ce
af
bf
cf

[2, 3]

" 2 → a, b, c
3 → d, e, f

res:
ad
ae
af

[2, 3]

a b c
[3] [3]
d e f
[] [] []

[2, 3, 5, 6, 7, 0, 1, 2, 5 - - - -]

```
void getPossibleWordsUtil(int i, vector<int> &arr, string &curr, vector<string> &result) {
    if (i == arr.size()) {
        result.push_back(curr);
        return;
    }

    for (char ch: keypad[arr[i]]) {
        curr.push_back(ch);
        getPossibleWordsUtil(i + 1, arr, curr, result);
        curr.pop_back();
    }
}
```

arr = [0, 2]

o/p = [a, b, c].

curr: ____

res =

```
void getPossibleWordsUtil(int i, vector<int> &arr, string &curr, vector<string> &result) {
    if (i == arr.size()) {
        result.push_back(curr);
        return;
    }

    for (char ch: keypad[arr[i]]) {
        curr.push_back(ch);
        getPossibleWordsUtil(i + 1, arr, curr, result);
        curr.pop_back();
    }
}
```

curr = ab

res: ad
ae
af

arr: [2, 3]

curr: [2, 3]

curr: [2, 3] curr: [1, 3] curr: [3, 3]

n = n.

TC = O(4^n)

n x 4^n

[2, 4, 6, 8], target = 8.

[2, 2, 2, 2] [2, 2, 4] [2, 6] [4, 4] [8].

8/2 = 2 8/2 = 2 - - - - - 8/min = 2x2x2x - - - = 2^3

([2, 4, 6, 8], 8)

exc: ([2, 4, 6, 8], 8)

inc: ([2, 4, 6, 8], 6)

[2 ____]

n (i, tar) tar
(i+1, 8) (i, tar-1)
(i+2, 8) (i+1, tar-1) (i, tar-2)

n = max (n, tar).

TC = 2^max(n, tar)

AS = max(max(n, tar), n, tar)
= max(n, tar)

D R D D R R.
D D R D R R

(i, j)
U → (i-1, j)
D → (i+1, j)
L → (i, j-1)
R → (i, j+1)

visited

1) In-Bound & the matrix
→ 2) It should be = 1

(0, -1) (-1, j) (i, j) (i, j+1) (0, +1)
(-1, j-1) (i+1, j)

(i, j)
(i-1, j) (i+1, j) (i, j-1) (i, j+1)
(i-1, j-1) (i+1, j-1) (i-1, j+1) (i+1, j+1)

n = n^n.

TC = O(4^n^2)
AS = O(n^2)

(0, 0)
(0, 1) (2, 2)
(0, 2)
(1, 2) (2, 1)
(1, 1)
(1, 0) → (2, 0)